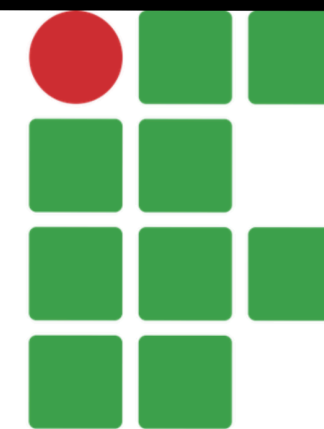




Orientação a Objetos



Exceções



**INSTITUTO
FEDERAL**
Acre



Professor

Me. Jonas Pontes



@jn.pontes

Introdução



- Em programação, temos os erros e as exceções;
 - Erros: são erros graves, não devemos tratar;
 - Exceções: erros recuperáveis



Introdução



- Uma **exceção** é uma indicação de um problema que ocorre durante a execução de um programa
 - é qualquer condição de erro ou comportamento inesperado encontrado por um programa em execução;
- O **tratamento de exceção** permite aos programadores criar aplicativos que podem resolver exceções.



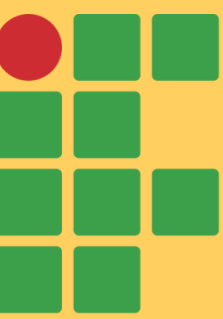
Introdução



- Em Java, uma exceção é um objeto herdado da classe:
 - `java.lang.Exception`-o compilador obriga a tratar ou propagar
 - `java.lang.RuntimeException`-o compilador não obriga a tratar ou propagar
- As exceções são lançadas quando um método detecta um problema e é incapaz de tratá-lo.



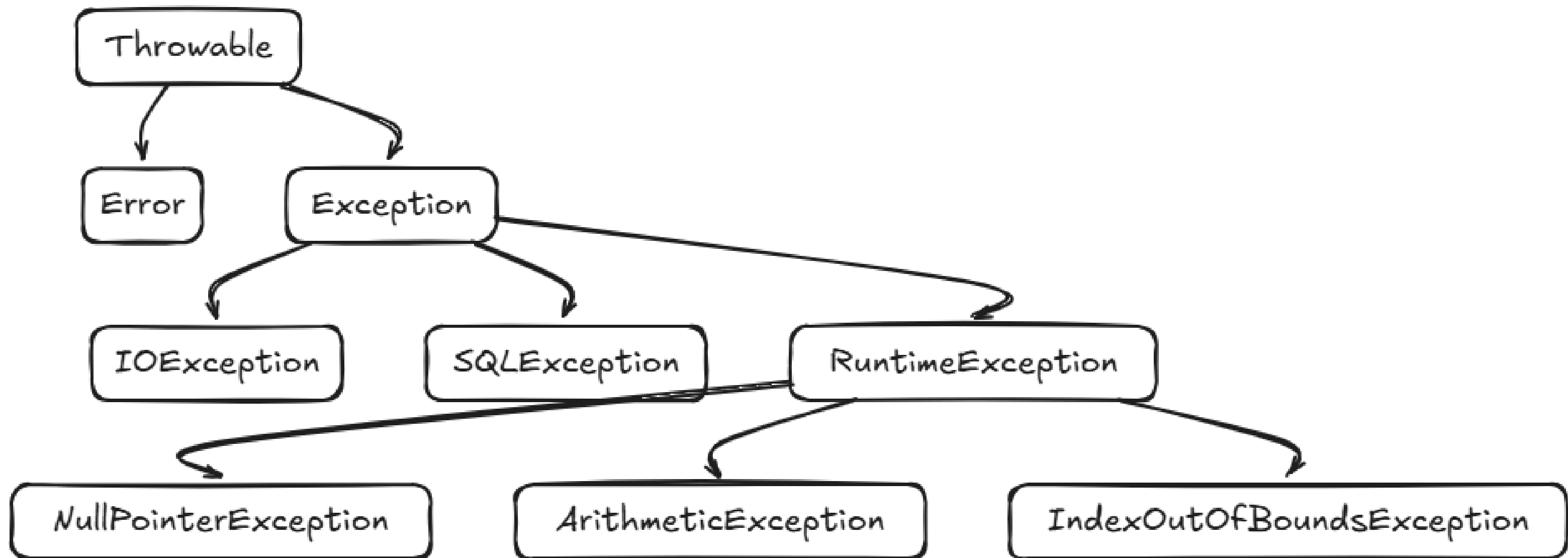
Exemplo



- O rastreamento de **pilha de uma exceção** inclui o nome da exceção em uma mensagem que indica o tipo de problema que ocorreu e a pilha completa de chamadas de método no momento em que a exceção ocorreu
 - Exemplo: `10/0` → `ArithmeticException`;
- O ponto no programa em que uma exceção ocorre é chamado de ponto de lançamento.



Hierarquia



Tipos de exceções



- Verificadas (checked)
 - São verificadas em tempo de compilação, como `IOException` e `SQLException`.
 - O compilador exige que você trate essas exceções com `try/catch` ou `throws`.



Tipos de exceções



- Não verificadas (unchecked)
 - Ocorrências comuns de erro lógico, são subclasses de RuntimeException.
 - O compilador não exige tratamento, mas é boa prática tratar.
 - Exemplo: NullPointerException, ArithmeticException.



try/catch



- Um **bloco try** inclui o código que talvez lance (throw) uma exceção e o código que não deve executar se essa exceção ocorrer;
- Bloco try
 - Contém o código que representa a execução normal do trecho de código que pode acarretar em uma exceção.



try/catch



- Um bloco catch inicia com a palavra-chave catch e um parâmetro de exceção seguido por um bloco de código que trata a exceção;
- Esse código executa quando o bloco try detecta a exceção;
- Bloco catch
 - Contém o código a ser executado caso uma exceção ocorra.



try/catch



```
try {  
    int r = 10/0;  
}  
catch (ExceptionType e) {  
    System.out.println(Erro: divisão por 0;  
}
```



try/catch



- Um bloco catch especifica entre parênteses um parâmetro de exceção identificando o tipo de exceção a tratar;
- Uma exceção não capturada é uma exceção que ocorre para a qual não há nenhum bloco catch correspondente;
- Uma exceção não capturada fará com que um programa termine antes da hora se esse programa contiver somente uma thread.



try/catch



- Se ocorrer uma exceção em um bloco try, o bloco try termina imediatamente e o programa transfere o controle ao primeiro bloco catch com um tipo de parâmetro que corresponde ao tipo de exceção lançada.



finally



- É um bloco que contém código a ser executado independentemente de ter ocorrido ou não uma exceção, como fechar um arquivo, conexão de banco de dados, ou outro recurso específico ao final do processamento.

```
try{ }  
catch (ExceptionType e){ }  
finally{ }
```



Cláusula throws



- Especifica uma lista separada por vírgula das exceções que o método pode lançar, e aparece após a lista de parâmetros do método e antes do corpo do método;
- Usado para propagar a exceção.



Cláusula throws



```
public void lerArquivo() throws IOException {  
    FileReader reader;  
    reader = new FileReader("arquivo.txt");  
}
```



Boas práticas



Trate exceções específicas antes das genéricas

- Prefira exceções mais detalhas (parte mais baixa da hierarquia de exceções);
- O tipo `Exception` pode:
 - Ocultar erros inesperados;
 - Tratar de forma genérica um problema que poderia ter uma solução melhor;
 - Dificultar o debug.





Evite capturar Exception diretamente

- Captura qualquer exceção, o que parece prático, mas pode esconder problemas;
- Use-a somente se:
- For realmente necessário capturar múltiplas exceções de uma vez.
- Está em um nível alto do sistema (ex: camada de controle), onde o objetivo é evitar a falha completa da aplicação.



Boas práticas



Evite capturar Exception diretamente

```
try {  
    // código que pode lançar várias exceções  
} catch (Exception e) {  
    System.out.println("Ocorreu um erro.");  
}
```



Boas práticas



- Use o bloco `finally` para fechar recursos:
 - arquivos ou conexões;
- Não ignore exceções:
 - ex: `catch (Exception e){}` vazio é má prática.



try/catch ou throws?



- Use try/catch quando:
 - Sabe-se como tratar o erro localmente;
 - O objetivo é proteger a execução de um trecho específico;
 - Está no nível de interface com o usuário ou no ponto final do processamento.



try/catch ou throws?



```
public void abrirArquivo() {  
    try {  
        FileReader reader;  
        reader = new FileReader("arquivo.txt");  
    } catch (FileNotFoundException e) {  
        System.out.println(  
            "Arquivo não encontrado.");  
    }  
}
```



try/catch ou throws?



- Use throws quando:
 - O método não deve tratar o erro, isso é responsabilidade de quem chamou;
 - Deseja-se propagar a exceção para que seja tratada em outro lugar;
 - Está escrevendo métodos utilitários ou de biblioteca, que não sabem o que fazer com o erro.



Exemplo comparativo



```
public void dividir(int a, int b) {  
    try {  
        int res = a / b;  
        System.out.println("Resultado: "+ res);  
    } catch (ArithmeticException e) {  
        System.out.println(  
            "Não é possível dividir por zero.");  
    }  
}
```



try/catch ou throws?



```
public int dividir(int a, int b) throws ArithmeticException {  
    return a / b;  
}  
  
public void executarDivisao() {  
    try {  
        int r = dividir(10, 0);  
    } catch (ArithmeticException e) {  
        System.out.println("Erro na divisão: " + e.getMessage());  
    }  
}
```

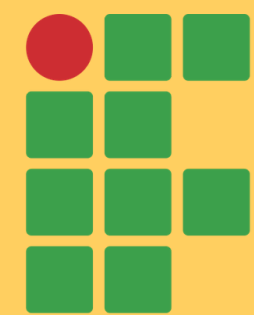




Trate a exceção no lugar onde faz mais sentido lidar com o erro.

Situação	Onde tratar
O método sabe como reagir	No próprio método
O erro depende do contexto (ex: interface gráfica vs. console)	No método que chama
O método é uma biblioteca ou serviço	Propague com throws e trate mais acima





Vamos para a prática

